
Cyber Security

SIS: Modern Encryption Algorithms/Using a Hacker Tool

Fullname: Amangeldi Zhanserik

ID: 22B030301

E-mail: zha_amangeldi@kbtu.kz

Date of submission: April 5, 2025

Class time: Thursday, 15:00–18:00



School of Information System and Engineering
Kazakh-British Technical University
Academic Year 2024-2025

Contents

1	Lab 1: Use Classic and Modern Encryption Algorithms	2
1.1	Use a Classic Encryption Algorithm	2
1.1.1	Launch the CSE-LABVM	2
1.1.2	Open and explore the JCrypTool	2
1.1.3	Use the Caesar algorithm to encrypt a text message	3
1.1.4	Decrypt the encrypted ciphertext with the Caesar algorithm.	4
1.1.5	Change the Caesar algorithm settings.	4
1.2	Use a Modern Symmetrical Encryption Algorithm	5
1.2.1	Use AES encryption to encrypt a text message.	5
1.2.2	Use AES to decrypt a text message.	6
1.3	Use a Modern Asymmetrical Encryption Algorithm	7
1.3.1	Use RSA asymmetrical encryption to encrypt a text file.	7
1.3.2	Use RSA asymmetrical encryption to decrypt a text file.	8
2	Lab 2: Encrypting and Decrypting Data using a Hacker Tool	9
2.1	Create and Encrypt Files	9
2.1.1	Create text files	9
2.1.2	Zip and encrypt the text files.	9
2.2	Recover Encrypted Zip File Passwords	10
2.2.1	Introduction to fcrackzip	10
2.2.2	Recovering Passwords using fcrackzip	10

1 Lab 1: Use Classic and Modern Encryption Algorithms

1.1 Use a Classic Encryption Algorithm

1.1.1 Launch the CSE-LABVM

I launched the CSE-LABVM using VirtualBox and logged in with my credentials.

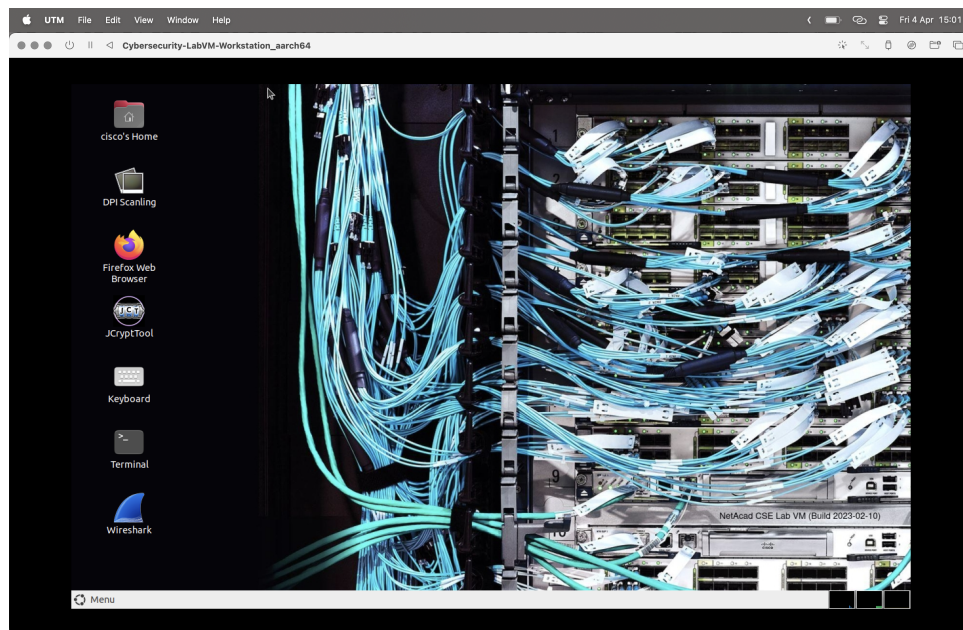


Figure 1: CSE-LABVM login screen

1.1.2 Open and explore the JCryptTool

I opened the JCryptTool application and explored its main windows and features.

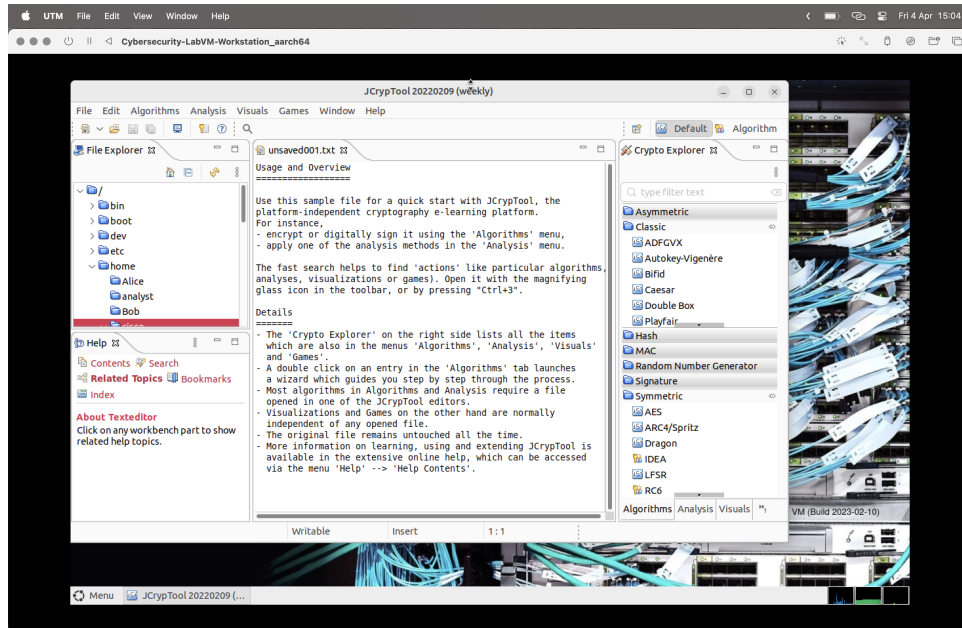


Figure 2: JCryptTool main screen

1.1.3 Use the Caesar algorithm to encrypt a text message

I used the Caesar algorithm to encrypt the text message "CRYPTOGRAPHY IS FUN. CAN YOU READ THIS SECRET MESSAGE?" with a JCryptTool.

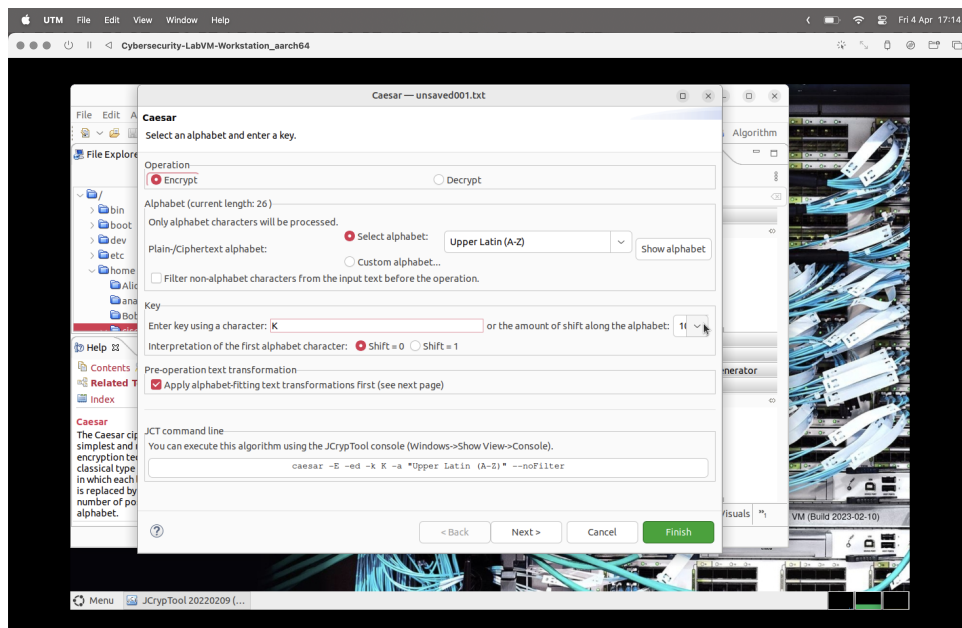


Figure 3: JCryptTool Caesar algorithm configuration

The encrypted message was: MBIZDYQBKZRI SC PEX. MKX IYE BOKN DRSC COMBOD WOCCKQO?

1.1.4 Decrypt the encrypted ciphertext with the Caesar algorithm.

I decrypted the ciphertext using the same Caesar algorithm with a same configuration, but chose the "Decrypt" operation.

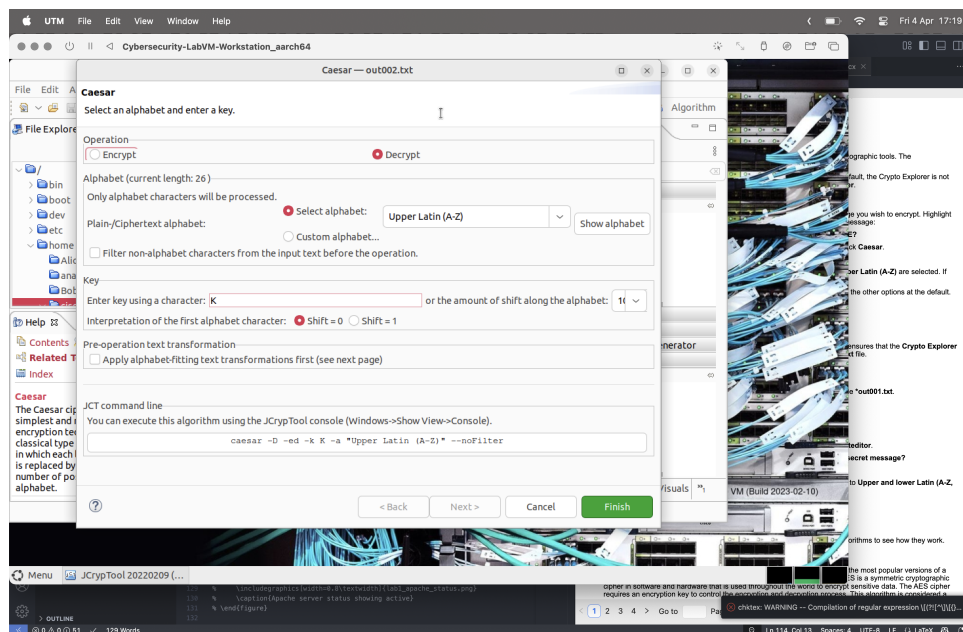


Figure 4: JCryptTool Caesar algorithm configuration for decryption

And yes in result I got the original message: CRYPTOGRAPHY IS FUN. CAN YOU READ THIS SECRET MESSAGE?

1.1.5 Change the Caesar algorithm settings.

For the last step I changed the Caesar algorithm configuration and enter new message(the same but with lowercase letters): Cryptography is fun. Can you read this secret message?

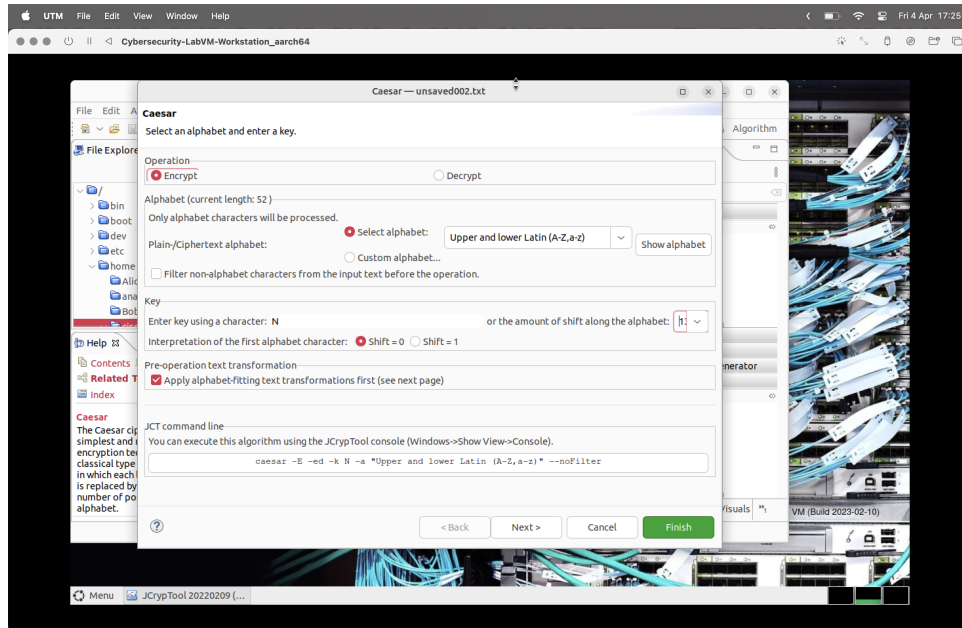


Figure 5: JCryptTool Caesar algorithm with changed configuration

The encrypted message was: PELCGBtEnCuL vF sHA. PnA LBH Ernq GuvF FrpErG zrFFntr?

1.2 Use a Modern Symmetrical Encryption Algorithm

In this part, we will use a modern symmetrical encryption algorithm, AES(Advanced Encryption Standard).

1.2.1 Use AES encryption to encrypt a text message.

I used the advanced algorithm to encrypt the text message "Cryptography is fun. Can you read this secret message?" using Symmetric(AES) encryption.

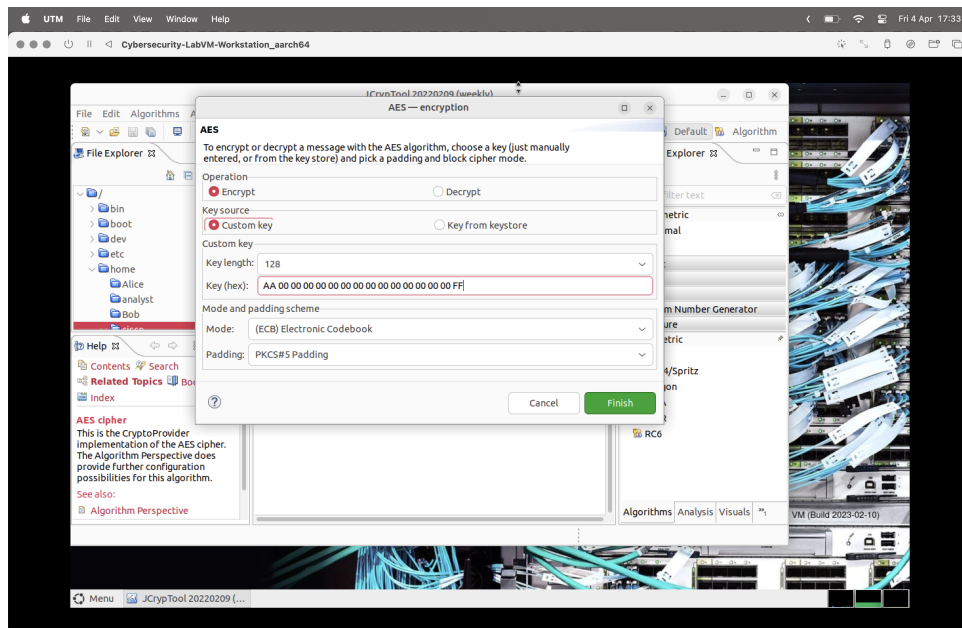


Figure 6: JCryptTool Symmetric AES configuration

The encrypted message was:

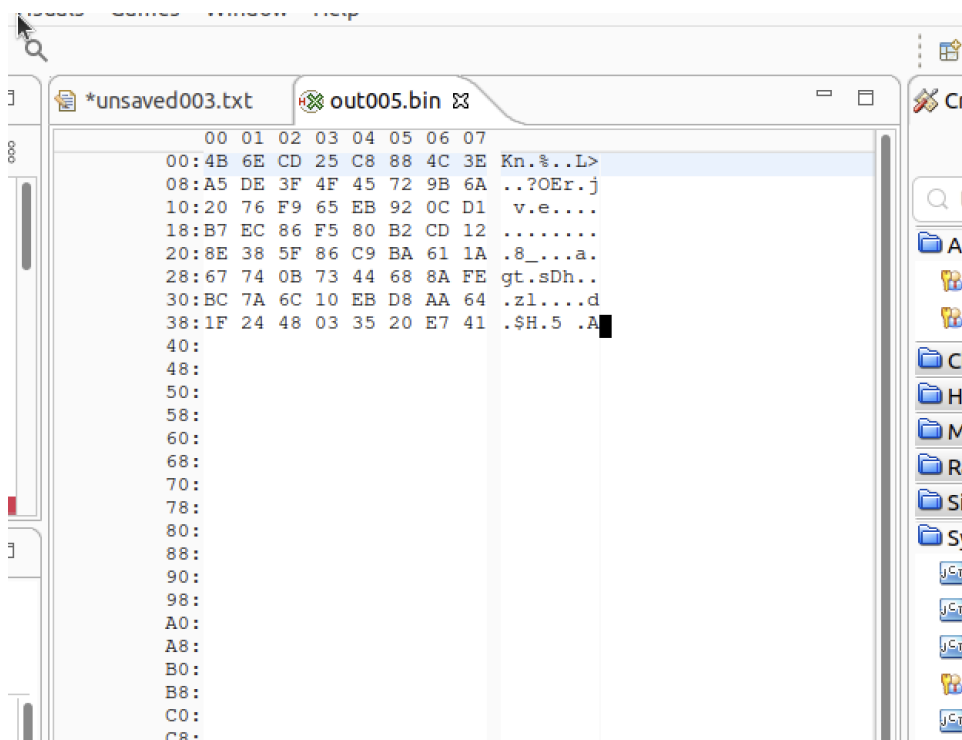


Figure 7: JCryptTool Symmetric AES result

1.2.2 Use AES to decrypt a text message.

For decrypting the message. I used the same AES algorithm's configuration, but chose the "Decrypt" operation and yes for result I got the same text message encrypted.

1.3 Use a Modern Asymmetrical Encryption Algorithm

In this part, we will use a modern asymmetrical encryption algorithm. Unlike symmetric encryption, asymmetric encryption encrypts and decrypts the data using two separate yet mathematically connected cryptographic keys. These keys are known as a 'Public Key' and a 'Private Key'.

1.3.1 Use RSA asymmetrical encryption to encrypt a text file.

There we used again the same message for encryption "Cryptography is fun. Can you read this secret message?", but with Asymmetrical(RSA) algorithm.

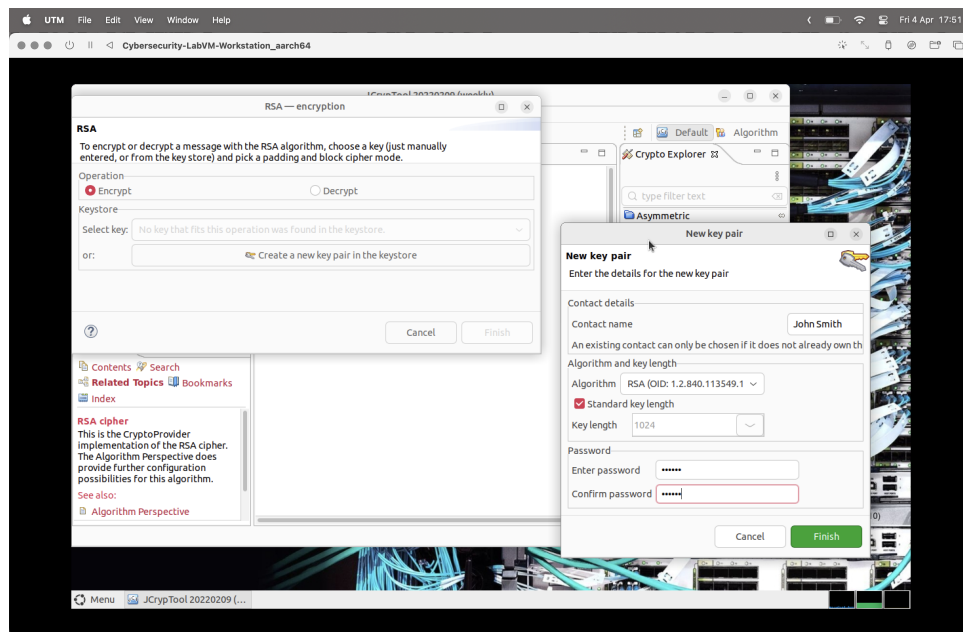


Figure 8: JCryptTool Asymmetric RSA configuration(public/private key)

And for result I got the encrypted message:

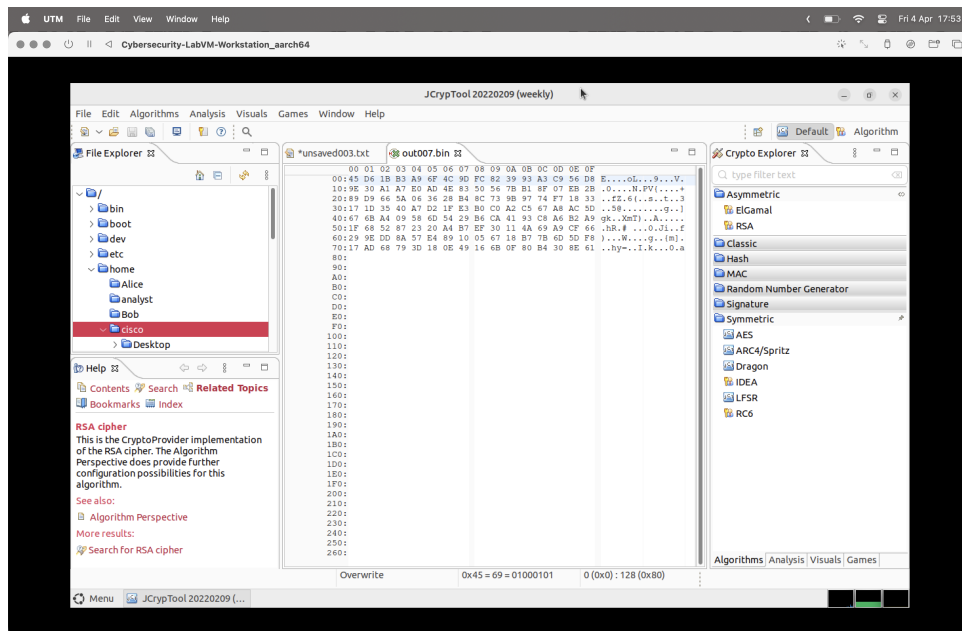


Figure 9: JCryptTool Asymmetric RSA result

1.3.2 Use RSA asymmetrical encryption to decrypt a text file.

I used the private key to decrypt the encrypted message. I used the same key that was generated during the encryption process.

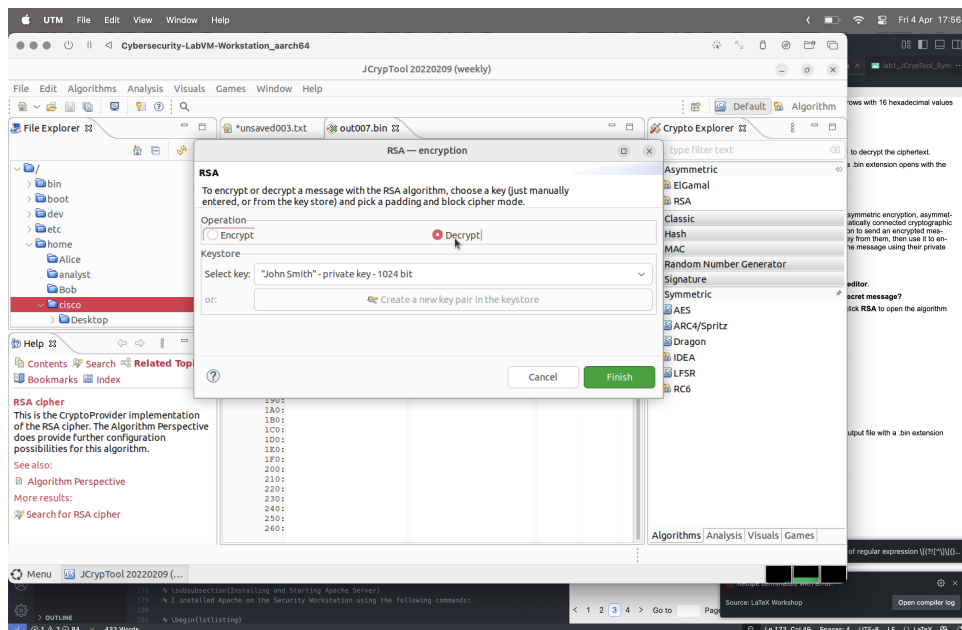


Figure 10: JCryptTool Asymmetric RSA decryption

and for result I got the same text message "Cryptology is fun. Can you read this secret message?".

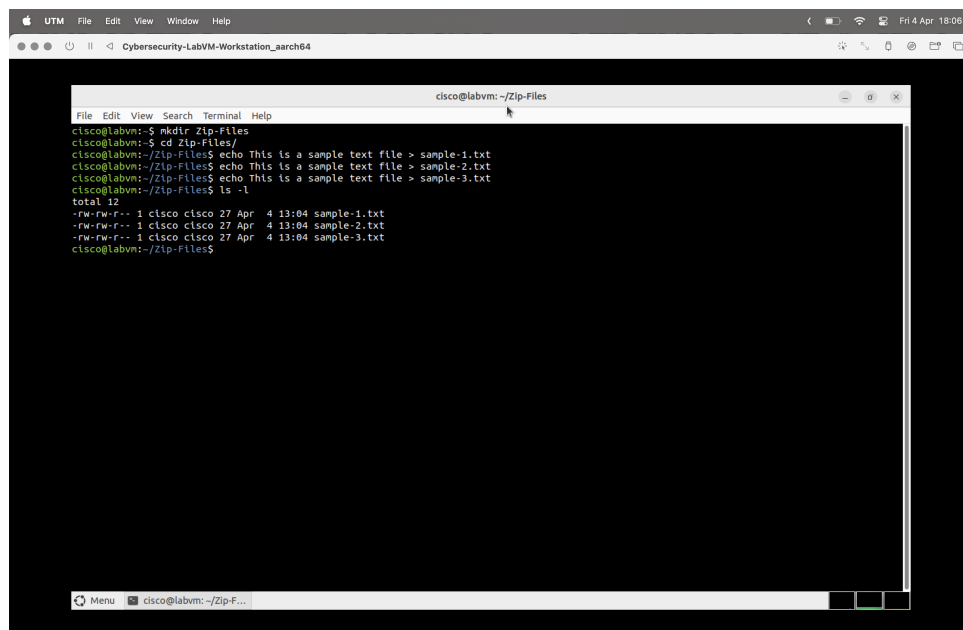
2 Lab 2: Encrypting and Decrypting Data using a Hacker Tool

2.1 Create and Encrypt Files

In this part, I will create a few text files that will be used to create encrypted zip files in the next step.

2.1.1 Create text files

I created three text files with the same text message "This is a sample text file" in dir Zip-Files.

A screenshot of a terminal window titled 'cisco@labvm: ~/Zip-Files'. The terminal shows the following commands and output:

```
cisco@labvm:~$ mkdir Zip-Files
cisco@labvm:~$ cd Zip-Files/
cisco@labvm:~/Zip-Files$ echo This is a sample text file > sample-1.txt
cisco@labvm:~/Zip-Files$ echo This is a sample text file > sample-2.txt
cisco@labvm:~/Zip-Files$ echo This is a sample text file > sample-3.txt
cisco@labvm:~/Zip-Files$ ls -l
total 12
-rw-rw-r-- 1 cisco cisco 27 Apr  4 13:04 sample-1.txt
-rw-rw-r-- 1 cisco cisco 27 Apr  4 13:04 sample-2.txt
-rw-rw-r-- 1 cisco cisco 27 Apr  4 13:04 sample-3.txt
cisco@labvm:~/Zip-Files$
```

Figure 11: Created three txt files

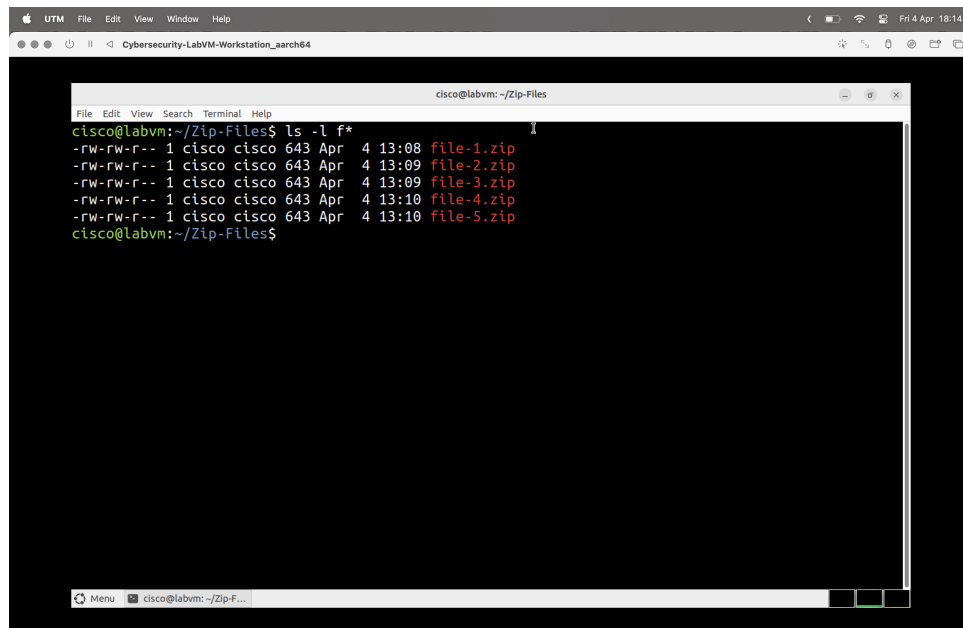
2.1.2 Zip and encrypt the text files.

There I created 5 zip files with different passwords, I used the same text files for zipping, and command was: `zip -e file-N.zip sample-*`

And passwords were:

- file-1.zip: B
- file-2.zip: R2
- file-3.zip: 0B1
- file-4.zip: Y0Da
- file-5.zip: C-3P0

in result I got 5 zip files with different passwords:

A screenshot of a terminal window titled 'cisco@labvm: ~/Zip-Files'. The terminal shows the command 'ls -l f*' being executed, which lists five files: file-1.zip, file-2.zip, file-3.zip, file-4.zip, and file-5.zip. Each file is owned by 'cisco:cisco', has permissions '-rw-rw-r--', and a size of 643 bytes. The files were created on April 4th at 13:08, 13:09, 13:09, 13:10, and 13:10 respectively. The prompt 'cisco@labvm:~/Zip-Files\$' is visible at the bottom.

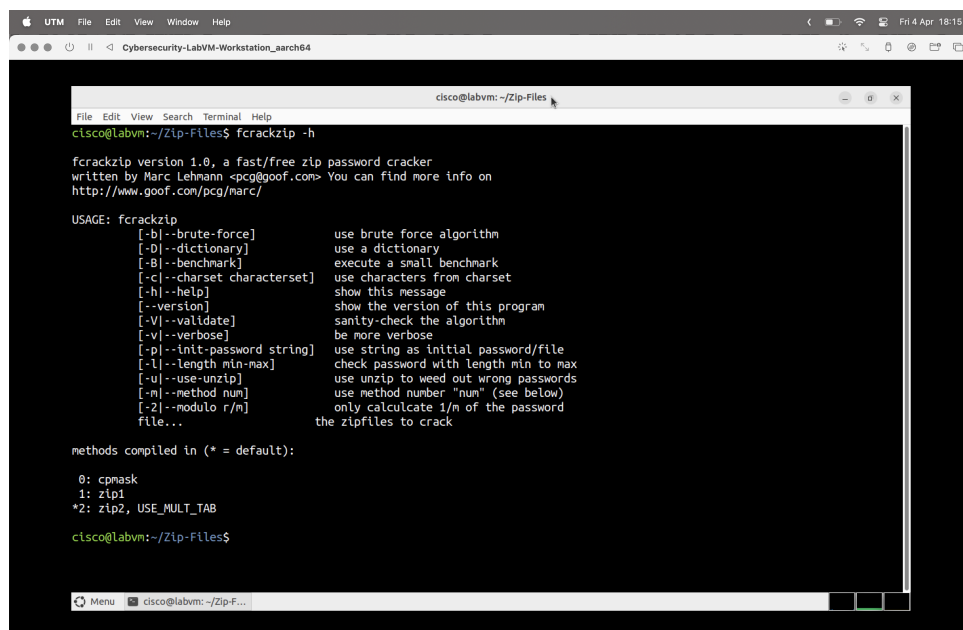
```
cisco@labvm:~/Zip-Files$ ls -l f*
-rw-rw-r-- 1 cisco cisco 643 Apr  4 13:08 file-1.zip
-rw-rw-r-- 1 cisco cisco 643 Apr  4 13:09 file-2.zip
-rw-rw-r-- 1 cisco cisco 643 Apr  4 13:09 file-3.zip
-rw-rw-r-- 1 cisco cisco 643 Apr  4 13:10 file-4.zip
-rw-rw-r-- 1 cisco cisco 643 Apr  4 13:10 file-5.zip
cisco@labvm:~/Zip-Files$
```

Figure 12: Created 5 zip files

2.2 Recover Encrypted Zip File Passwords

2.2.1 Introduction to fcrackzip

For this part, I check the new terminal command for cracking zip file passwords. The command is fcrackzip.

A screenshot of a terminal window titled 'cisco@labvm: ~/Zip-Files'. The terminal shows the command 'fcrackzip -h' being executed, which displays the help text for fcrackzip. The text includes the version (1.0), author (Marc Lehmann), and a list of usage options such as [-b]--brute-force, [-D]--dictionary, [-B]--benchmark, [-c]--charset, [-h]--help, [--version], [--validate], [--verbose], [--init-password string], [--length min-max], [--use-unzip], [--method num], and [-2]--modulo r/m. It also lists methods compiled in: 0: cpnask, 1: zip1, and *2: zip2, USE_MULT_TAB.

```
cisco@labvm:~/Zip-Files$ fcrackzip -h
fcrackzip version 1.0, a fast/free zip password cracker
written by Marc Lehmann <pcg@gooof.com> You can find more info on
http://www.gooof.com/pcg/marc/

USAGE: fcrackzip
        [-b]--brute-force      use brute force algorithm
        [-D]--dictionary      use a dictionary
        [-B]--benchmark       execute a small benchmark
        [-c]--charset          use characters from charset
        [-h]--help            show this message
        [--version]           show the version of this program
        [--validate]          sanity-check the algorithm
        [--verbose]           be more verbose
        [-p]--init-password string use string as initial password/file
        [-l]--length min-max  check password with length min to max
        [-u]--use-unzip        use unzip to weed out wrong passwords
        [-m]--method num      use method number "num" (see below)
        [-2]--modulo r/m      only calculate 1/m of the password
        file...               the zipfiles to crack

methods compiled in (* = default):

0: cpnask
1: zip1
*2: zip2, USE_MULT_TAB

cisco@labvm:~/Zip-Files$
```

Figure 13: fcrackzip command list

2.2.2 Recovering Passwords using fcrackzip

In this part we will use the fcrackzip command to recover passwords for the zip files we created in the previous parts. For all files except file-5.zip I used the command(in 5th

file we use 1-5 instead of 1-4): `fcrackzip -vul 1-4 file-N.zip`
and I got this result for all files(how much time it took to crack the password):

- file-1.zip: 0.11 seconds
- file-2.zip: 0.6 seconds
- file-3.zip: 0.38 seconds
- file-4.zip: 1.108 seconds
- file-5.zip: 50.194 seconds

For calculation I used the Bash-Script:

Listing 1: bash version

```
#!/bin/bash

# Input from command line
ZIP_FILE=$1

# Start time in milliseconds
START_MS=$(date +%s%3N)

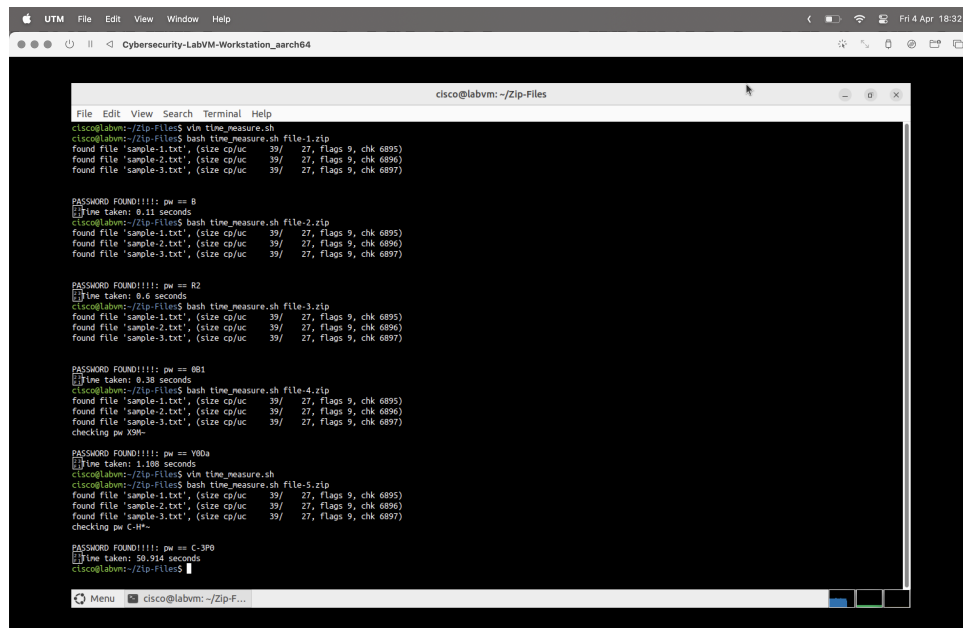
# Run fcrackzip, adjust the range as needed
# Here we are using 1-4 for the first four files
fcrackzip -vul 1-4 "$ZIP_FILE"

# End time in milliseconds
END_MS=$(date +%s%3N)

# Duration in milliseconds
DURATION_MS=$((END_MS - START_MS))

# Convert to seconds with milliseconds
SECONDS=$((DURATION_MS / 1000))
MILLISECONDS=$((DURATION_MS % 1000))

echo "Time_taken: ${SECONDS}.${MILLISECONDS}seconds"
```



```
cisco@labvm: ~/Zip-Files
File Edit View Search Terminal Help
cisco@labvm:~/Zip-Files$ time measure.sh
cisco@labvm:~/Zip-Files$ bash time_measure.sh file-1.zip
found file 'sample-1.txt', (size cp/uc 39/ 27, flags 9, chk 6895)
found file 'sample-2.txt', (size cp/uc 39/ 27, flags 9, chk 6896)
found file 'sample-3.txt', (size cp/uc 39/ 27, flags 9, chk 6897)

PASSWORD FOUND!!!!: pw == 0
time taken: 0.0 seconds
cisco@labvm:~/Zip-Files$ bash time_measure.sh file-2.zip
found file 'sample-1.txt', (size cp/uc 39/ 27, flags 9, chk 6895)
found file 'sample-2.txt', (size cp/uc 39/ 27, flags 9, chk 6896)
found file 'sample-3.txt', (size cp/uc 39/ 27, flags 9, chk 6897)

PASSWORD FOUND!!!!: pw == R2
time taken: 0.11 seconds
cisco@labvm:~/Zip-Files$ bash time_measure.sh file-3.zip
found file 'sample-1.txt', (size cp/uc 39/ 27, flags 9, chk 6895)
found file 'sample-2.txt', (size cp/uc 39/ 27, flags 9, chk 6896)
found file 'sample-3.txt', (size cp/uc 39/ 27, flags 9, chk 6897)

PASSWORD FOUND!!!!: pw == 001
time taken: 0.38 seconds
cisco@labvm:~/Zip-Files$ bash time_measure.sh file-4.zip
found file 'sample-1.txt', (size cp/uc 39/ 27, flags 9, chk 6895)
found file 'sample-2.txt', (size cp/uc 39/ 27, flags 9, chk 6896)
found file 'sample-3.txt', (size cp/uc 39/ 27, flags 9, chk 6897)
checking pw X0M-

PASSWORD FOUND!!!!: pw == Y00a
time taken: 1.108 seconds
cisco@labvm:~/Zip-Files$ bash time_measure.sh file-5.zip
found file 'sample-1.txt', (size cp/uc 39/ 27, flags 9, chk 6895)
found file 'sample-2.txt', (size cp/uc 39/ 27, flags 9, chk 6896)
found file 'sample-3.txt', (size cp/uc 39/ 27, flags 9, chk 6897)
checking pw C-H-

PASSWORD FOUND!!!!: pw == C-3P0
time taken: 59.914 seconds
cisco@labvm:~/Zip-Files$
```

Figure 14: Result of cracking zip files

For interest I also tried to crack the password with 6 Characters(how said in lab), and for that I created file-6.zip with password JarJar. First I thought that it will take 30-40 minutes approximately, but it took 5315,617 seconds, which is about 1.5 hours. The last question was about what I recommend a password needs to be for it to be secure, and I think that password should be at least 8 characters long and contain a mix of characters, including uppercase and lowercase letters, numbers, and special characters. This makes it more difficult for attackers to guess or crack the password using brute force methods.